本章路線

這章要回答的核心問題是：在磁碟、網路儲存與 SSD 等不同裝置上，如何安排 I/O 與冗餘以符合延遲、吞吐與故障保證？

前置知識。 block 與 sector、I/O interrupt、機率與平均延遲。

- 建立 disk service-time 模型
- 比較 FCFS、SSTF、SCAN、C-SCAN
- 分析 RAID level 0/1/4/5/6 的可靠性與寫入成本

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Media機制決定I/O成本，介面數字不等於實測

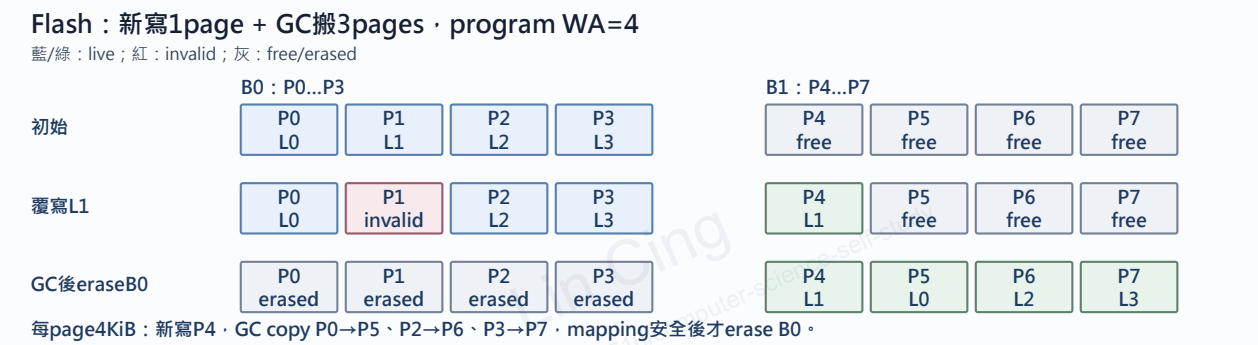
HDD以磁性表面、旋轉platter與移動head存取；SSD以nonvolatile semiconductor媒體配合controller/FTL；tape適合串流與較長定位成本的備份/封存。Host看到logical blocks，不直接控制每個physical位置。比較需固定request size、random/sequential、read/write比例、queue depth、cache與同步契約。

指標	明確分母	不能混用
Bandwidth	完成bytes / elapsed seconds	media/interface peak不必等end-to-end
IOPS	完成requests / seconds	4KiB和1MiB request的IOPS不可直接比吞吐
Service time	裝置實際服務區間	不含尚未開始的queue wait
Response latency	arrival至完成/回報	視契約可含排程、queue、copy
Durability	授權成功後可承受的故障	I/O completion不一定已離開volatile cache

固定每秒1000個4KiB完成requests，bandwidth=3.90625MiB/s；改為1000個64KiB則62.5MiB/s。相同IOPS不代表相同bytes/sec，也不能由上述算術推論任何實際HDD/SSD產品速度。

Flash out-of-place update與GC的真實工作量

固定NAND模型page4KiB、erase block4pages=16KiB。初始B0的P0/P1/P2/P3分別存L0/L1/L2/L3；B1的P4…P7皆erased。覆寫L1時寫新P4、mapping L1→P4，舊P1變invalid，不能直接在未erase的P1任意覆寫。



若回收B0，仍live的L0/L2/L3需分別copy到P5/P6/P7並更新mapping，確認live資料與必要mapping安全後才能erase B0。Host新寫1page，physical program=1new+3GC copies=4pages，這個crash-free計數模型write amplification=4。若L2先被discard且不再需要，GC只copy2pages，program總3pages、WA=3；不計mapping寫入等其他開銷。

Discard/TRIM告知某些logical資料不再需要，不保證立刻physical erase或可當安全抹除。Wear leveling分散erase負擔；GC搬移、over-provisioning與controller cache也會改延遲。若加入power loss，還要證明FTL mapping與資料commit順序，不能只靠這張正常路徑圖。

Cylinder/sector與LBA分屬不同抽象

傳統HDD的同一半徑tracks集合稱cylinder，seek改半徑，rotation使目標sector到head下方；sector/track幾何可因zone與缺陷remap不同。LBA是一維logical block編號，不應把LBA直接當永遠固定的physical cylinder；SSD更沒有移動磁頭可供SSTF最佳化。

固定sector512bytes，LBA4096對應logical byte offset4096×512=2MiB。若介面改4KiB logical block，同一編號對應16MiB，單位不可省。32-bit無號LBA可表示2^32個512-byte blocks，容量2^41bytes=2TiB；這是位址空間算式，不是所有partition/API限制的共同答案。

若上層一次寫512bytes而底層原子/physical單位4KiB，可能需要read-modify-write或裝置轉換；alignment影響效能與crash粒度。實際保證必須查device/driver contract，不能由filesystem block size推定atomic sector size。

磁碟容量：把幾何與單位逐層展開

假設磁碟組有 10 片雙面 platter，最上和最下各一個外表面不存資料，因此可用面數為 `10×2−2=18`。每面有 2,048 條 track，每條 track 有 4,096 個 sector，每個 sector 為 8,192 bytes；所有面採相同幾何，不另扣備援或格式化空間。

$$18 \times 2{,}048 \times 4{,}096 \times 8{,}192 = 1{,}236{,}950{,}581{,}248 \text{ bytes}.$$

同一數量除以 2^{30} 是 **1,152 GiB**；除以 2^{40} 是 **1.125 TiB**。若使用十進位 `1 GB = 10⁹ bytes`，則約 **1,236.95 GB**。標示單位才能比較數字；這是給定幾何下的容量，並非格式化後一定可用的容量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從轉速與每圈資料量推算時間

假設 6,000 rpm、每條 track 1,024 個 sector、每 sector 4 KiB；相鄰 sector 連續可讀，忽略 sector gap、controller、快取與排隊成本。

步驟	計算	結果
每秒圈數	6,000 ÷ 60	100 圈／秒
每圈時間	1 ÷ 100	0.01 秒 = 10 ms
每圈資料	1,024 × 4 KiB	4 MiB
理想循序速率	100 × 4 MiB／秒	400 MiB/s
平均旋轉等待	目標相位均勻，半圈	5 ms

若 seek 固定為 3 秒，接著讀取在同一 track 上連續排列的 4 MiB，從所需 sector 開始讀完恰需一圈，即 `4 MiB ÷ 400 MiB/s = 0.01 秒`。平均總服務時間是 `3 + 0.005 + 0.01 = 3.015 秒`。這是指定模型的算例；實際磁碟的幾何、gap、快取與其他開銷都可能不同。

Lin Cing

7200RPM與64KiB讀取：每段時間分開算

明定教學HDD參數：seek5ms，7200RPM且目標相位均勻，transfer128MiB/s，request64KiB，無cache hit與額外controller成本。每圈60,000/7200=25/3ms；平均rotation=25/6ms；transfer=64KiB/(128MiB/s)=125/256ms。

Service=5+25/6+125/256=7415/768≈9.65495ms。Queue wait若15ms，總response≈24.65495ms；單request串列、無重疊時IOPS≈1000/9.65495=103.574。不能把多queue並行、cache或同一track連續requests套入此倒數。

循序傳1GiB的純transfer下界為1024MiB/(128MiB/s)=8s，定位與queue另加。隨機小I/O的定位比重與大串流不同，正是不能僅拿介面bandwidth評估體感latency的原因。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Host-attached、NAS與SAN：哪裡解析檔案

路徑	Client送出的抽象	Filesystem metadata在哪裡處理	額外故障邊界
Host-attached block	本機driver/device block requests	本機host filesystem	controller/device
NAS	遠端file/directory操作	file server	network、server、cache/重試
SAN block	網路/存儲fabric上的logical blocks	使用該block device的host	fabric、target、multipath

SAN不等於遠端file API；NAS也不是只把本機磁頭接長線。若兩hosts把同一SAN LUN當獨占本機磁碟同時mount不具協調的filesystem，metadata可能互相破壞；shared block可見不等於已提供distributed locking/一致性。

固定4KiB read：NAS client送file handle/offset/length，server查filesystem再送block I/O；SAN client先在本機filesystem把file offset轉LBA，再由block transport送target。兩者可能讀同一實體array，卻在pathname、cache與失敗重試上有不同責任。

Multipath與共享故障域

兩條fabric路徑若都接同一target/controller，能改善部分link failure，不能假設有兩份獨立資料。固定paths A/B，A斷而B與target存活可切路；target失效則兩條路都不可用。Retry必須遵守request identity與write completion契約，不能在不知先前是否提交時任意重做非冪等上層操作。

同樣地，remote mount成功只證當時能完成相關服務，不證所有後續I/O可用或持久。第17章處理network partition/replication語意；Latency由request size、protocol、network、cache與queueing共同決定，不由local/NAS/SAN名稱唯一決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

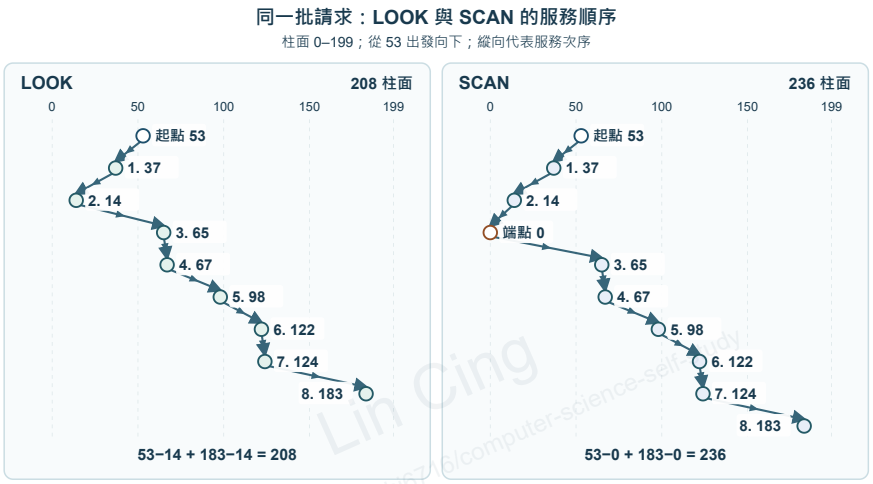
相同batch的磁頭移動：先固定端點與計費

固定cylinders0…199，head53，所有requests於t0到齊：98,183,37,122,14,124,65,67。成本只算相鄰服務/轉向位置的絕對cylinder差，直到最後請求完成；不含rotation、transfer或最後空跑至另一端。SSTF同距離取較小cylinder。

Policy	含轉向/回繞的路徑	總移動
FCFS	53→98→183→37→122→14→124→65→67	640
SSTF	53→65→67→37→14→98→122→124→183	236
LOOK往下	53→37→14→65→67→98→122→124→183	208
SCAN往下	53→37→14→0→65→67→98→122→124→183	236
C-SCAN往上	53→65→67→98→122→124→183→199→0→14→37	382
C-LOOK往上	53→65→67→98→122→124→183→14→37	322

C-SCAN/C-LOOK回繞也算physical移動，但不在回程服務requests；若別題明定不計回繞成本，必須另列計費而不是偷換本表數值。

SCAN的0與LOOK的14不能混成208



0 是 SCAN 的實體端點，不是請求；兩條路線都在最後請求 183 完成時停止。

LOOK下行到最後請求14就折返：|53−14|+|183−14|=39+169=208。SCAN本題必先到physical端點0再折返：|53−0|+|183−0|=53+183=236。SSTF恰好也236只是此輸入的巧合，規則與路徑並不相同。

FCFS逐差為45+85+146+85+108+110+59+2=640。SSTF每步距離12/2/30/23/84/24/2/59，加總236；每一決策只挑當時pending最近者，不保證整個batch總移動全域最小。LOOK208已是本例反證。

相同請求，起始方向會改變路線

固定 cylinder 0–199、起點 53；前一位置為 50，並明定本例初始朝較大的 cylinder 前進。八個請求均在開始前到齊：`98，183，37，122，14，124，65，67`。每段費用為移動距離，直到最後一筆請求完成即停止；不含旋轉或資料傳輸。

方法	路徑（199 是轉向點，不是請求）	各段距離	總移動
SCAN 向上	53→65→67→98→122→124→183→199→37→14	12、2、31、24、2、59、16、162、23	331
LOOK 向上	53→65→67→98→122→124→183→37→14	12、2、31、24、2、59、146、23	299

SCAN 在此約定到實體端點 199 才反向；LOOK 在該方向最後一筆請求 183 即反向。這兩條路徑都不是循環掃描，沒有額外回繞。若改以本章另一例的**向下**方向起步，SCAN 與 LOOK 分別是 236 與 208；同一組請求的數值不能脫離起始方向、端點及停止條件使用。

等待與效能保證依arrival/service模型

有限固定batch沒有無限starvation，SSTF的starvation風險來自新requests可持續在附近抵達，使遠端一直不被選。SCAN/LOOK方向掃描降低此偏好，但有限等待仍依bounded device range、每圈服務量與新到達的admission/每track處理規則；無限湧入同一位置不應被默認可在有限時間服務完。

Real device可在controller內重新排程、request merging與parallel queues；SSD沒有seek半徑，所以最小cylinder移動不直接代表最佳latency。選policy還要看deadline/fairness與throughput；表中成本只是明列的HDD定位抽象。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partition、logical formatting與mount是不同操作

Low-level/physical格式安排媒體sector及錯誤偵測等device細節，常由device製造/管理；partition在logical disk切位址範圍；logical formatting建立filesystem metadata/free-space structures；mount把可識別filesystem接到namespace。格式化不等於把所有舊資料安全抹除，也不等於自動mount。

固定1024MiB教學disk、sector512bytes，保留[0,1)MiB作抽象boot/partition metadata；A=[1,101)MiB長100MiB，B=[101,301)MiB長200MiB。A first LBA2048、sector count204800、exclusive end206848；B first206848、count409600、exclusive end616448；範圍不重疊。

這不是完整GPT layout規格，只驗證partition單位、左閉右開邊界與不同filesystem對象。A格式化/資料驗證失敗，不因B能mount而變成功；mount到同一路徑也不證底下是相同device/UUID。Bootloader能找kernel與kernel能mount root還需第2章的早期driver依賴。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bad blocks、remap與修復不改物件身分契約

Controller可把壞physical sector重新對應spare，host仍看到同LBA；這可能保住之後可用性，卻不保證已無法讀出的舊bytes可恢復。資料checksum與冗餘可協助偵測/重建，但錯誤必須向上保留，不能把「已換到spare」當作檔案內容已正確。

Boot block受損可能讓loader找不到kernel；filesystem metadata受損可能讓檔名/配置不可重建；data block損壞則可能只影響部分內容。Recovery/backup要對到相應層次與已知版本，而不是看到某個partition存在就直接宣稱整顆disk已修復。

Swap space是頁面backing，不是通用持久檔案

Anonymous頁面可能需要swap backing；clean file-backed pages可從原file重讀，未必另寫swap。Swap partition與swap file是配置/存取路徑選擇，實作可預建extent mappings降低正常filesystem查找成本；不能把歷史swap-file路徑成本當所有新版本的必然結論，也沒有無條件RAM兩倍的容量公式。

固定4KiB頁與swap slot S7，內容41，兩個private-COW邏輯頁暫以S7作backing。下表只算本模型未消失的swap-PTE references，不冒充Linux實際swap_map編碼或所有swapcache refs。

事件	P1	P2	S7仍需服務的swap引用
初始	swap S7	swap S7	2
P1讀回並私有寫42	private resident F23=42	swap S7，應讀41	1
P2讀回	private F23=42	resident F24=41	0
無in-flight I/O/cache保留引用	各自resident	各自resident	才能回收S7

P1新值不能覆寫仍被P2引用的S7=41；必要時為P1的新版本另配backing。此處不固定Linux的fault次數或共用swapcache page策略，只驗證引用/內容/lifetime的必要性。

Slot回收必須等最後使用者，不只看某個process退出

如果還有swap PTE、in-flight device request或保留的cache ownership，提早把slot給另一頁可讓後續fault讀到別人的資料。Refcount模型需把計數範圍、誰增減、何時I/O停止與哪些旗標另計明列；單說「有兩個process，所以count=2」不是所有實作的規格。

Swap資料一般用於當次系統執行的記憶體延伸，不是備份；hibernate映像則有額外恢復契約。加swap可改善commit餘裕，但若active working set超過physical frames，可能造成thrashing與長latency，不能用容量取代第9章的locality/回收分析。

Striping、mirroring與parity的容量/故障條件

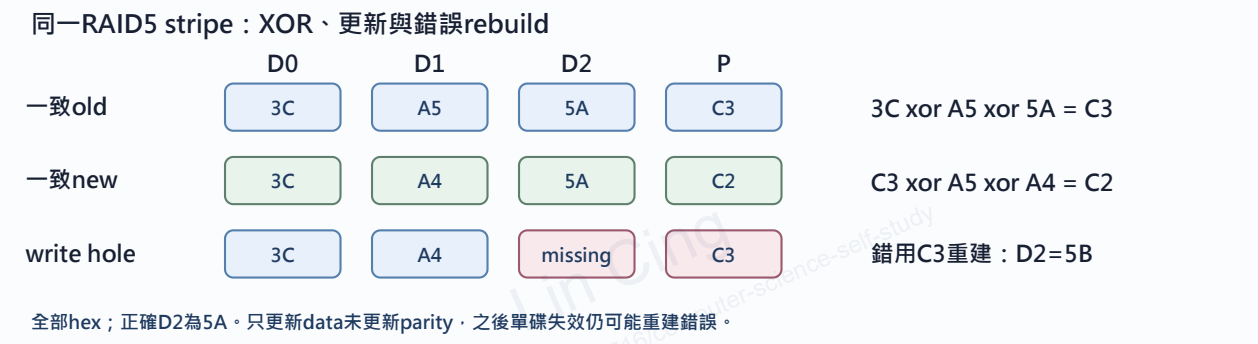
固定N顆同容量D，忽略metadata/spares；stripe是資料配置，mirror/parity提供特定故障下冗餘。RAID不是backup，刪除、惡意加密與共同controller錯誤仍可能同步破壞多份。

Level	組合概念	可用容量	固定完好array的device-loss保證
0	stripe無冗餘	ND	無冗餘，任一所需disk失效可失資料
1雙副本	pair mirror	每pair D	每pair可少1；不同pair與同pair雙失敗不同
2	bit striping與ECC	依code/check disks	依所選ECC與故障假設，不亂填通用公式
3	fine-grained stripe+dedicated parity	(N−1)D	1disk；多disk常參與同request
4	block stripe+dedicated parity	(N−1)D	1disk；parity disk可成write瓶頸
5	block stripe+distributed parity	(N−1)D	任意1disk，parity位置逐stripe分散
6	兩個獨立parity relations	(N−2)D	合格code/layout下任意2disk

RAID1可由多mirror組或一組多副本構成，容量先看拓樸。4disk的兩個2disk mirrors合計2D；一個四副本mirror只有D，不能共用同一容量答案。

RAID5一個stripe的XOR與小寫入

固定一個stripe有3個data bytes D0=3C、D1=A5、D2=5A，parity P=3C xor A5 xor 5A=C3（全部hex）。D1失效可由C3 xor3C xor5A=A5重建；單一XOR只提供一條關係，若兩個data同時未知則不足以唯一決定。



將D1改A4，P'=P xorD1 xorD1'=C3 xorA5 xorA4=C2。Cold uncached小寫入的read-modify-write需讀old D1與P、寫new D1與P'，共2reads+2writes；不含journal/cache額外I/O。完整3data stripe寫入可直接算P'並做4writes，無需為這個parity先讀old stripe。

Write hole會讓之後重建出錯

若new D1=A4已落盤但parity仍C3，此時stripe內仍能直接讀到D0/D1/D2，卻不再滿足parity。接著D2失效，錯用old parity重建得C3 xor3C xorA4=5B，而正確D2是5A。能容忍「任意一disk failure」的前提包含冗餘一致，不包含任意中斷造成的dirty parity再加disk loss。

可用具合適持久保護的write-back cache、parity/journal協議或copy-on-write等方式保護更新；其commit/恢復規則必須真正實作。僅加mirror、checksum或一個RAM flag不自動修復。重建期間要讀其餘devices，會帶來額外負載與再次failure風險，不能把rebuild speed當原始foreground bandwidth。

RAID6的第二條關係為何足夠

在適當finite field上，P=ΣD_i，Q=Σα_iD_i，兩失效位置a/b的係數α_a≠α_b；此處加法為field加法，在binary field可用XOR。扣除已知項後有A+B=P_missing，α_a A+α_b B=Q_missing。

代入A=P_missing+B，得(α_a+α_b)B=Q_missing+α_a P_missing。因係數不同，α_a+α_b非零可逆，故B唯一，再得A。這是兩個獨立parity relations的代數理由；若第二份只是複製同一XOR，兩式相依而仍不足。實際code要明定field多項式、layout與錯誤/erasure模型。

具體選GF(2^8)、modulus0x11D、係數1/2/4，資料3C/A5/5A。Q=3C xor(2・A5) xor(4・5A)=3C xor57 xor75=1E。失D1/D2後P_missing=FF、Q_missing=22，(2+4)B=22 xor(2・FF)=22 xorE3=C1；6的inverse為7A，B=7A・C1=5A，再A=FF xor5A=A5。乘法為此field的carryless/reduction操作，不是普通整數乘法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

RAID 6：逐類檢查兩個失效位置

固定一個 stripe，每個符號位於不同磁碟；資料符號為 D_i，$P=\oplus D_i$，$Q=\oplus(\alpha_i D_i)$。運算在 GF(2^8) 中進行，加法就是 XOR，係數 α_i **非零且兩兩不同**。下表假設失效位置已知、其餘符號正確，且同一 stripe 最多失去兩個符號。

失效符號	使用的資訊與恢復方法
D_a、D_β	從 P、Q 各扣除已知資料，解兩個方程；矩陣行列式 $\alpha_a \oplus \alpha_\beta$ 非零，解唯一。
P、D_a	從 Q 扣除其餘資料，得 $\alpha_a D_a$，再乘以 α_a^{-1}；最後重算 P。
Q、D_a	從 P 扣除其餘資料，得到 D_a；最後重算 Q。
P、Q	所有資料仍在，直接重算兩個 parity。

非零係數確保「P 與一筆資料同失」可由 Q 恢復；係數相異確保「兩筆資料同失」的兩個方程獨立。這是已知位置的 erasure 保證，不能據此推論能辨識任意靜默損壞，或容忍超出模型的共同故障。

P 與 D_1 同時遺失：用 Q 找回資料

採用多項式 0x11D 定義 GF(2^8)，各運算元以兩位十六進位表示。資料 D_0=3C、D_1=A5、D_2=5A，係數依序為 01、02、04，原本 P=C3、Q=1E。

P 與 D_1 遺失，但 D_0、D_2、Q 可讀。先從 Q 扣掉已知資料貢獻；在此 field 中扣除等同 XOR：

$$02D_1 = Q \oplus D_0 \oplus (04D_2) = 1E \oplus 3C \oplus 75 = 57.$$

因 02 的乘法反元素是 8E，故 D_1=8E·57=A5；最後重算 P=3C⊕A5⊕5A=C3。檢查 02·A5=57、04·5A=75，即可核對 Q。乘法採 carryless 運算並對 0x11D 約簡，不是一般整數乘法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

0+1與1+0不能只看有一半冗餘

固定4disks。RAID0+1先stripe(A,B)、stripe(C,D)，再mirror兩stripe；RAID1+0先mirror(A,B)、mirror(C,D)，再stripe兩pairs。A失效後，0+1只剩完整(C,D)stripe可用；1+0則B供應第一pair，C/D供應第二pair。

此拓樸下再失C：0+1兩stripe皆有缺件而失效；1+0兩pairs仍各有B/D而可用。再失B則相反：0+1的(C,D)仍完整，1+0的(A,B)pair全失。兩種布局的容錯集合不同，不能不畫分組就說任意兩碟都安全。

固定同期間每disk獨立loss機率p、無repair，雙mirror存活率$1-p^2$；四disk RAID5為(1−p)^4+4p(1−p)^3。Common power/firmware/環境與rebuild會破壞獨立無repair假設；公式不是裝置實測壽命保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Checksum樹與pool配置：偵測/冗餘/配額各有責任

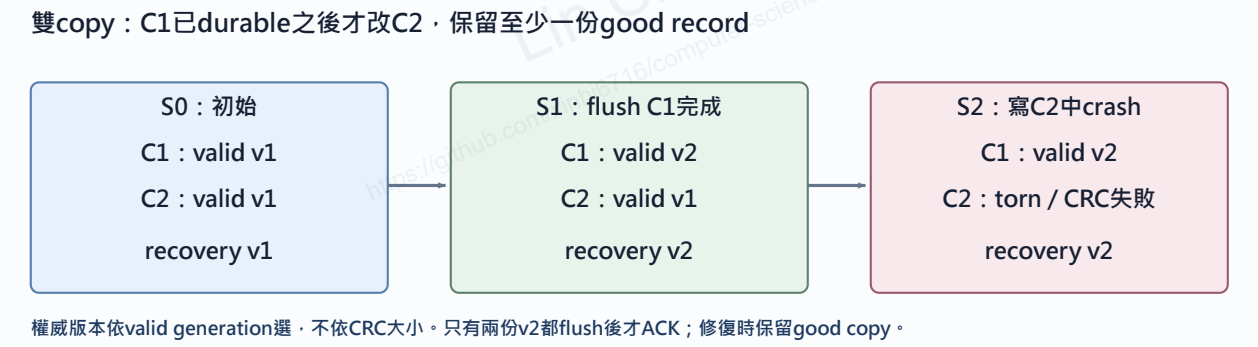
本書ZFS案例在parent metadata存child地址與checksum，沿樹驗證metadata/data；若只在同一壞block裡自存一個也可被一同改壞的checksum，不能等同已知可信上層的end-to-end驗證。Checksum可偵測指定模型的損壞，只有另有正確冗餘來源才可修復，且checksum不等於對惡意者的authentication。

Storage pool把底層配置供多filesystem動態取用，quota限制某filesystem增長，reservation保障指定份額。固定抽象100GiB pool，A用30、B用20、free50；A quota60，B reservation40尚需保護20，則未受保留free30，A最多再用min(quota餘30,未保留30)=30。這是排除snapshot/dedup/compression的配額模型，不冒充所有ZFS accounting細節或無容量上限。

Stable storage是故障模型下的保證

理想stable storage不遺失資訊；實作只能在明列故障範圍下近似。固定serialized logical512-byte record，兩份device copies，generation不wrap，CRC可偵測本例注入的torn writes；durable flush可信。每一時刻只覆寫一個copy；本模型的單次crash／torn write只影響正在改寫的copy，另一未被改寫的copy及已完成flush的copy保持有效。同時破壞這些未寫／已flush copies的共同故障不在本次不變量證明內。CRC不是對任意corruption無碰撞證明。

Record格式為4-byte big-endian generation、504-byte payload、4-byte CRC32（覆蓋前508bytes）。v1 payload為ASCII V1加502zero bytes，CRC=BF4CA2E8；v2同法V2，CRC=4A112E80。Recovery只比較通過checksum的records，再選最高generation；不能拿CRC數字大小當版本順序。



每個crash prefix留下哪個完整版本

Crash邊界	copy1	copy2	可恢復結果
初始	valid v1	valid v1	v1
寫copy1途中、未確認flush	v1/v2/torn	valid v1	至少v1；若完整v2可取v2
copy1 v2已flush，尚未寫copy2	valid v2	valid v1	v2
寫copy2途中	valid v2	v1/v2/torn	v2，不是v1
copy2 v2也flush，之後ACK	valid v2	valid v2	ACK的v2可恢復

寫copy1時copy2保留old valid；第一份new durable之後才動copy2，所以第二階段copy1保留new valid。這個不變量由初始兩valid歸納維持至少一份valid。Recovery先保留所選good copy，再修另一份且flush；即使修復中再crash，good copy仍在。若兩份同時更新、flush不可信或雙份共同lost，前提破壞，不能套此證明。

Logical operation在ACK前crash可能恢復old或完整new，是未回覆操作的允許結果；不能把「未ACK」解釋成必然沒執行。單record的保證也不自動讓多record filesystem transaction atomic。

Flush、FUA、排序與atomicity分開

Volatile write-back cache可在資料未落nonvolatile媒體前回completion。Flush要求指定先前寫入達持久層；FUA要求該request的完成符合持久承諾；兩者是否由device原生支援、模擬及如何跨remapping layers傳遞，依driver契約。現代Linux文件的REQ_PREFLUSH/REQ_FUA是具體例，不能當成OSC9所有歷史裝置都有相同flag名稱。

CPU memory barrier不等於device cache flush，write ordering不等於單次write不會tear；單record雙copy亦不同於RAID5 parity一致性。必須把host buffer、device cache、persistent media與ACK逐層連起，才知道哪種power loss可以恢復。

同一份資料的定位、冗餘與持久完成

HDD排程的53→14或53→0改變定位成本；SSD FTL把logical L1映到新physical page並搬移live資料；RAID用mirror/parity在指定故障下重建；stable-write協議決定更新途中crash留下何版本。四者可能同時存在，不可用其中一層成功替代另一層保證。

本章數例已分出SCAN236與LOOK208、RAID5 C3/C2及錯誤重建5B、雙copy更新第二階段必保留v2。每個結論都可由表內狀態/位址/資料重新計算，不只記policy名稱。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

評估成本與恢復時保持同一分母

將queue wait、seek/rotation、transfer與scheduler delay分開；計IOPS時同時標request bytes；計write amplification標host與physical bytes；計RAID可用容量先畫拓樸；談可靠度標故障域/相關性與repair；談ACK標flush/atomicity前提。

將HDD改SSD時，排程應重新考慮queue、並行度與flash搬移成本；改RAID拓樸時，重新列每個failure set能否重建；改flush或cache策略時，重新檢查ACK前後可恢復的版本。資料錯誤偵測、重建來源與歷史備份都須各自存在。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 10.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study